

Agenda

- Introduction
- Datatypes in Kotlin
- Functions and their types
- Higher Order functions and Closures
- Null Safety
- Class
- Extension Function
- Data class

What is Kotlin

- It is a programming language given by JetBrains
- JetBrains have developed softwares like Android Studio, IntelliJ Idea, Pycharm, etc
- It is new improved version of Java
- It can work with JVM environment as it is a general purpose language
- It is developed to target multiple platforms
- In 2011, JetBrains announced the development of Kotlin Language
- In 2016 Kotlin 1.0 was made live
- It is made as open source
- In 2017 Google announced the development of android apps with the use of kotlin language
- In 2019, Google announced the Kotlin as the primary language for the Android Application Development

Features of Kotlin

- It is Statically Typed Language
- It is an OOP Language
- It is a Functional language
- It is 100% interoperable with Java
- Concise(Lines of code is less), Safe(Less errors at runtime) and Powerful
- Open Source

How Kotlin is built on top of Java

- Uses the JVM as its runtime
- Uses the Java class file format
- Compiles .kt → .class files
- Produces Java bytecode
- Uses same garbage collector as Java
- Uses same JIT and HotSpot optimizations
- So Kotlin is basically is a language that compiles to the same bytecode as Java, but with a cleaner and safer syntax.

JVM versions Kotlin that supports

Kotlin Version	JVM Bytecode Target	Notes
Kotlin 1.0 → 1.3	Java 6	Earliest Android support
Kotlin 1.4 → 1.7	Java 8	Default
Kotlin 1.8 → 2.x	Java 11	Recommended
Kotlin 2.x+	Java 17 / 21	LTS support (2024+)

- So Kotlin supports, Minimum Java: 6
- But today, Recommended target: Java 11 or Java 17

Development Environment

- Java (atleast JDK 8)
- Requires an IDE for developemnt (IntelliJ Idea Community Edition)

Variables

- Their are two types of variables in Kotlin

1. var

- It can be reassigned

2. val

- It cannot be reassigned

```
fun main() {  
    var n1 = 10 // similar to let in JS  
    n1 = 100 // OK  
  
    val n2 = 20 // similar to const in JS  
    //n2 = 200 // NOT OK  
}
```

DataTypes

1. Integer (Byte, Short, Int, Long)
2. Floating Point (Float, Double)
3. Boolean (True, False)
4. Character (Char, String)

- In Kotlin the datatypes are automatically detected depending on what type of data is assigned to the variables
- It is caled as type inferenece
- If a variable is assigned with a value of some type initially and if we try to change the value with some differnt type then compiler gives error.
- This feature is called as type checking. It checks for compatibility
- We can also provide the Data Types Explicitly

```
fun main() {  
    var n1 = 10 // Int (type is inferred) -> type inference  
    //n1 = true // Once a variable is initialized with a specific type its type cannot  
    be changed  
  
    var n2: Int // Explicit data type  
    n2 = 20  
    println("n2 - " + n2)  
  
    var n3: Double = 30.0  
    var n4: Float = 40f  
    var n5: Boolean = true  
    var n6: Char = 'A'  
    var n7: String = "Sunbeam"  
  
}
```

Functions

- Block of code to perform, certain functionality
- Helps to reuse code

```
fun add(parameter1:type,parameter2:type):returnType{  
    // body  
}
```

Types of Function

1. Parameterless and Parameterized

- mentioning datatype to the parameter is compulsory.
- function parameters are of val type i.e we cannot change/reassign the value of the parameters inside the function body.

```
//Parameterless function  
fun f1() {  
    println("Inside f1")  
}  
  
//Parameterized function  
fun f2(n1: Int, n2: Int) {  
    println("n1 = ${n1},n2=${n2}")  
}  
  
fun f3(n1: Int, n2: Int) {  
    // n1 = 100 // we cannot change the data inside the parameters as they are of  
    val type
```

```
println("Inside f3 - ($n1 , $n2)")  
}
```

2. Overloaded Function

- Same name different signature where signature can be changed by changing
 1. No of parametrs
 2. Types of parametrs
 3. Order of Parameters

```
fun f2(n1: Int, n2: Int) {  
    println("n1 = ${n1},n2=${n2}")  
}  
  
//Overloaded function  
fun f2(n1: Int) {  
    println("n1 = ${n1}")  
}
```

2. Default Argument Function

- we can provide default values to the parameters of the function, these functions are called as default argument function.

```
fun f3(n1: Int = 1, n2: Int = 2, n3: Int = 3) {  
    println("n1 = ${n1},n2=${n2},n3=${n3}")  
}
```

3. Anonymous and Inline Function

- We can store a function inside a variable, this variable is of a type called as function type
- If their is only 1 statement i.e only return statement inside the function then the function can be made as inline

```
// sq1 variable is called as a variable of function type  
// the function without name is called anonymous function  
val sq1 = fun(n: Int): Int {  
    return n * n  
}  
  
//inline function  
fun sq2(n: Int) = n * n
```

```
//Anonymous inline function
val sq3 = fun(n: Int) = n * n
```

Lambda Expression

```
// Lambda expression
val sq5 = { n: Int -> n * n }

// lambda expression with function type
val sq6: (Int) -> Int = { n -> n * n }
```

NamedArguments

- We can provide names to the arguments while passing which will map with the parameter name.
- this is called as named arguments

```
fun display(id: Int, name: String, salary: Double) {
    println("Id - ${id}")
    println("Name - ${name}")
    println("Salary - ${salary}")
}

fun main() {
    display(1, "Anil", 10000.0);

    // named arguments
    display(name = "Mukesh", salary = 20000.0, id = 2)
}
```

New Datatypes in Kotlin

1. Unit

- used for functions just like void
- If function does not return any thing then by default its returntype is Unit

```
// function returns Unit
fun f1(n1: Int, n2: Int) {
    println("Inside f1 - ($n1 , $n2)")
}

// function that returns Unit with no value
fun f2(): Unit {
    println("Inside f2 that returns Unit")
    return
}
```

```
println("fun f2 finished")
}
```

2. Nothing

- used for functions that can only throw exception but cannot have return statement

```
fun f3(): Nothing {
println("Inside f3, return type is Nothing, try returning before finish")
// return // cannot return from Nothing function
throw Exception("From F3") // Only Exception can be thrown
println("f3 finished")
}
```

3. Any

- used to store any type of data in the variable or to return any type of data from function

```
fun f4(): Any {
println("fun f4 that returns anything")
// return 10 // OK
// return 11.22 // OK
// return true // OK
return "sunbeam" // OK
}

fun main() {
var variable: Any // It means any type of Datatype
variable = 10
println("variable value - $variable")
variable = 11.22
println("variable value - $variable")
variable = true
println("variable value - $variable")
variable = "sunbeam"
println("variable value - $variable")
}
```

Higher Order Functions & Closures

- A Higher-Order Function (HOF) is any function that does one of these:
 - Takes another function as a parameter (accepts a lambda or function reference)
 - Returns a function (gives back a lambda or function reference)
- A closure is a function (usually a lambda) that remembers the variables from the outer scope even after that scope is finished.
- A closure = function + captured variables.

```
//Higher Order Function
fun executer(fn: (n1: Int, n2: Int) -> Int) {
    val res = fn(20, 10)
    println("Result = ${res}")
}

// Closure
fun callApi(): () -> String {
    var count = 0
    val token: String = "asdfhksfkdbsfkibfjklebnjkfsbnf";
    return { // this lambda is a closure
        count++
        "Mytoken : ${token}-${count}"
    }
}

fun main() {
    executer(fun(n1: Int, n2: Int): Int = n1 + n2)
    executer({ n1: Int, n2: Int -> n1 - n2 })
    executer { n1, n2 -> n1 * n2 }

    val fn = callApi()// The function execution finished here local p2.variable is destroyed
    println(fn()) // Still we can get the incremented value as lambda preserved it.
    println(fn()) // Still we can get the incremented value as lambda preserved it.
    println(fn()) // Still we can get the incremented value as lambda preserved it.
    // this is called as closure
}
```

Null Safety

- Null safety is a Kotlin feature designed to significantly reduce the risk of null references, also known as The Billion-Dollar Mistake
- One of the most common in many programming languages, including Java, is that accessing a member of a null reference results in a `NullPointerException`, or an NPE for short.
- Kotlin explicitly supports nullability as part of its type system, meaning you can explicitly declare which variables or properties are allowed to be null.
- Also, when you declare non-null variables, the compiler enforces that these variables cannot hold a null value, preventing an NPE.
- Nullable types and non-nullable types are as below

```
var s1: String = "sunbeam" // Non Nullable Type
// s1 = null; // cannot be made as null as its declared non nullable
```

```
var s2: String? = "sunbeam" // Nullable Type
s2 = null // can be made as null as its declared nullable

println(s1.length) // guranted their won't be null pointer exception as its non nullable

// println(s2.length) // chances are their will can be null pointer exception as its nullable
```

- There are several ways to handle nullables:

1. Check for null with the if conditional
2. Safe call operator ?.
3. Elvis operator ?:
4. Not-null assertion operator !!

```
// 1. If condition
if (s2 != null)
    println("Checking if condition - ${s2.length}")

// 2. safe call operator
println("safe call operator - ${s2?.length}") // returns null if s2 is null but no NPE

// 3. Elvis Operator
println("Elvis operator - ${s2?.length ?: 0}") // returns 0 if s2 is null

// 4. Not-null Assertion Operator
println("Not-Null Assertion operator - ${s2!!.length}") // NullPointerException if s2 is null (For Java Developers)
```

Class

- class is defined using a class keyword
- class consists of properties and methods
- object is created by using the class name

```
class Person(var name: String, var age: Int) // properties
{
    //method
    fun canVote(): Boolean {
        return age > 18
    }
}

fun main(){
    var p1 = Person("p1", 21) // Object
```



```
p1.age = 28
p1.canvote()
}
```

Data Class

- In java we use pojo classes which is used only to store data that can be used for transferring data accross multiple layers.
- for this we need to provide the fields and all setters and getters, i.e we need to write the boiler plate code multiple times.
- this is avoided in kotlin with the help of data classes.
- the data classes are defined using the keyword `data`
- these classes helps us to avoid writing the boiler playte code of getter,setters, toString, equals and hashCode method.
- we can also destructuure our data classes which is also called as component n functions

```
data class Person(var id:Int,var name:String) {
}

fun main() {
    val p1 = Person(1,"Rohan")

    // helps to create an object copy of data classes
    val p2 = copy(p1)
    val p3 = p1.copy(id=3)
    println(p2)
    println(p3)

    //this is called as destructuring the data class
    // this is also called as component n functions
    val(id,name) = p3
    println(id)
    println(name)

    // this represents the no of components we have inside our data classes
    println(p3.component1())
    println(p3.component2())
}
```

Extension Function

- We cannot add any functions inside the library classes.
- If we want to add our functions then we need to extend those classes
- In kotlin it is possible to add the functions inside the library classes without overriding them
- this functions are called as extension function